

Predicting Housing Market Prices in the Netherlands

Project 2-2 Report

Group 08

Marta Adamowska, Eira Gardner, Przemyslaw Grudka, Taha Koç,
Valerija Krumplevskaja, Nikitas Ttikou, Ignacio Zendrera Bosch

2025

Abstract

Accurate forecasting of residential property prices is crucial for buyers, sellers, investors, and policymakers navigating the dynamic housing market of the Netherlands [1]. Starting from an initial dataset of 5,555 raw record. We evaluated the performance of three supervised learning models—Linear Regression [2], Random Forest [3], and XGBoost [4]—on this enriched dataset.

XGBoost achieved the best generalization performance ($R^2 \approx 0.72$, MAE $\approx$ EUR 108k), outperforming Random Forest ($R^2 \approx 0.68$, MAE $\approx$ EUR 112k) and Linear Regression ($R^2 \approx 0.55$, MAE $\approx$ EUR 128k). SHAP analysis confirmed that living area, property type, construction year, and lot size are among the most influential predictors, accounting for most of the model variance.

These findings highlight the effectiveness of gradient-boosted ensembles in housing price prediction, offering transparent and interpretable insights for real-time valuation. Future research should incorporate temporal market dynamics and more granular geographic features to further enhance prediction accuracy.

1 Introduction

The Dutch housing market, characterized by its controversial and economic impact on society, stemming

from several interconnected issues such as the affordability crisis [5], presents a great set of challenges followed by opportunities for a wide range of stakeholders. Recent years have seen drastic changes in home price, leading to widespread confusion, demotivation (exhaustion in the search for a home) [6] and significant inconsistencies in home valuation and selection [1]. An important aspect of this comes down to a reliable determination of residential property prices, which not only guides prospective buyers and sellers in the market but also facilitates investors seeking to make informed decisions and policymakers aiming to foster market stability and develop effective housing strategies. Fluctuations in housing prices can have consequences, influencing individual wealth investment returns and broader economic health. Therefore, the development of robust predictive models is a critical area of research to provide a well-defined set of methods that aid and guide interests, including the decision-making process for stakeholders.

An exciting approach to this problem is the use of Machine Learning methods for real estate prediction. While the use of machine learning is not a new method of real estate prediction globally, it is uncommon in its application in the Netherlands, as our area of interest. Existing automated appraisal models such as the Woz-waarde are preferred, although it is not optimal [7]. The Woz-waarde focuses on different aspects of valuation, such as taxation, rather than variables in market dynamics, and

ultimately lacks granularity for cross-city comparison. Hence, the exploration of Machine Learning approaches may offer a more effective way of identifying patterns and relationships that may be overlooked in traditional appraisal methods, providing stakeholders with a clearer representation of the value of the market [8].

This project aims to address this gap by developing and evaluating different models of supervised machine learning models, specifically Linear Regression, Random Forest, and XGBoost, and also experimenting with a Stacked and Gated model, to predict housing prices across the Netherlands, and also testing new datasets to test the models' performance. Leveraging a comprehensive, multi-city dataset comprising over 60,000 samples, our primary objective is to compare the predictive accuracy of these models. A key focus extends beyond performance to model interpretability by using techniques such as SHAP to identify the primary drivers of value. These findings will not only provide insight into the Dutch market, but are also designed to provide stakeholders with a reliable tool for model-based valuation and analysis.

1.1 Research questions

1. How do advanced ensemble models (Stacking, Mixture-of-Experts) compare to traditional models (XGBoost, Random Forest, Linear Regression) in predicting Dutch housing prices?
2. Does the feature selection of the SHAP model correctly represent the most influential factors in Dutch housing price predictions compared to baseline feature selection methods?
3. Which features have the highest impact on Dutch housing prices according to decision tree prediction?

2 Data Preprocessing

Before training the machine learning models, the data was transformed to improve its quality and ensure the correct operation of the algorithms.

2.1 Dataset Description

The dataset used in this study is the "Dutch House Prices Dataset"¹, which includes 5,555 real estate transaction records from the Netherlands. Each row corresponds to an individual residential property and contains both numerical and categorical attributes. The dataset is structured with 16 features describing each property's characteristics. The attributes include:

- **Address and City:** Exact street-level address and corresponding municipality.
- **Price:** The transaction price of the property, expressed in euros.
- **Lot and Living Space:** Total lot size and usable living space, both measured in square meters.
- **Build Information:** Construction year, type of building (e.g., new or existing), and structural category (e.g., semi-detached house).
- **Architectural Details:** Roof type and number of floors.
- **Room Information:** Total number of rooms and bathrooms.
- **Energy Efficiency:** Energy label ranging from A (efficient) to G (inefficient).
- **Location Attributes:** Proximity details like whether the house is in a quiet residential area.
- **Garden Description:** Whether the property includes a backyard, front yard, or both.
- **Estimated Neighborhood price per m²:** An auxiliary value estimating local pricing trends.

¹<https://www.kaggle.com/datasets/bryan2k19/dutch-house-prices-dataset/data>

2.2 Dataset Filtering

- **Handling missing values** The gaps in the columns were filled with the mode (the most frequent value) for categorical features or the median for numeric features. This ensures the stability of the model, prevents errors during training, and reduces the impact of outliers, especially in skewed distributions.
- **Cleaning and transforming the price** The values in the column "Price" contained currency symbols and spaces, which prevents them from being interpreted as numbers. These symbols were removed and the values themselves were transformed to the numeric type. This is critical since the price is the target variable of the model and must be quantitative.
- **Transforming numeric features** The columns "Total Rooms" and "Bedrooms" could contain non numeric values or gaps. To ensure correct model operation, they were converted to the numeric type and emptied of gaps using the median values.
- **Encoding of categorical features** The column "City" is encoded with LabelEncoder, as it is a nominal category with many unique values. Other categorical features such as "Build type", "House type", "Roof" were encoded with One-Hot Encoding, which allows the model to treat each category as a separate binary feature without creating an artificial order between them.

2.3 Data Analysis

Data optimization step: removing outliers

After the basic data preparation, an additional data quality improvement step was performed - removing the most expensive real estate objects, potential outliers. This step was performed to increase the accuracy of the predictive model.

Instead of removing all anomalous data, the most expensive objects were discarded one by one, and at each step the quality metric (the determination coefficient R^2) was estimated in the test sample (see Fig-

ure 1). This approach allowed us to empirically determine the optimal number of objects to remove, at which the model shows the best generalization quality. An increase in R^2 during this process suggests that the remaining data became more homogeneous, allowing the model to better capture underlying patterns.

As a result, after removing 30 most expensive properties, the model showed the best result ($R^2 = 0.5733$ for Linear Regression Model). The final sample became more balanced and typical for the mass market, which is especially important when building models focused on the middle segment of real estate.

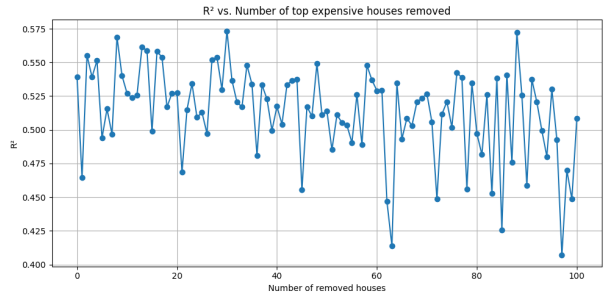


Figure 1: Relationship between model performance (R^2) and the number of removed high-price outliers.

3 Methods

3.1 Linear Regression

Linear Regression (LR) was employed as a baseline model due to its simplicity, interpretability. It assumes a linear relationship between independent input variables (predefined features) and a continuous dependent variable (target prices). The model estimates the coefficients for each feature in a way that minimizes the residual sum of squares between the actual and predicted values [2].

3.2 Random Forest

The Random Forest model constructs an ensemble of M decision trees, each trained on a bootstrap sample

of the training set with a random subset of features considered at each split. Predictions are obtained by averaging the outputs of the individual trees, which reduces variance while maintaining low bias. We set $M = 100$ and fix the random seed for reproducibility. Feature importance is computed from the mean decrease in impurity across all trees, enabling identification of the most predictive variables. Model evaluation follows the same 80/20 train-test split and metric suite as Linear Regression, with additional diagnostic plots generated to inspect residual patterns and prediction errors.

3.3 XGBoost

Extreme Gradient Boosting (XGBoost) is an advanced ensemble machine learning algorithm designed to handle complex and non-linear relationships within data effectively. It improves upon traditional gradient boosting by incorporating regularization terms and optimized computational performance, making it particularly suited for large datasets like ours.

For our implementation, we utilized the XGBoost library available in Python, leveraging its built-in capabilities for parallel processing and efficient training. The model was trained using the same 80/20 training-test split described previously, with hyperparameters tuned through a cross-validation strategy to avoid overfitting. Key hyperparameters such as learning rate, maximum tree depth, and the number of estimators were systematically adjusted using GridSearchCV to identify the optimal configuration.

3.4 A Stacked Model

In machine learning, stacked models are an ensemble technique that combine several traditional models into one big model. The idea of using a stacked model is to combat the various cons of each model while utilising the pros as well. A stacked model works by fitting the data to a set of base models, in our case we used Linear Regression and Random Forest, then a final model will take the predictions of the base models and use them to calculate a final

prediction, for our model our final estimator was an XGBoost model [9].

3.5 SHAP

Additionally, we integrated SHapley Additive exPlanations (SHAP) to interpret the models predictions. SHAP values provided insight into the individual contribution of each feature towards predicted housing prices, enabling transparent and intuitive explanations of model outputs. This interpretability step was essential to validate that the model aligned with domain knowledge and to ensure trustworthiness in practical applications.

3.6 Neural Mixture-of-Experts

We assemble a small, diverse team of “expert” models—say, a tree-based regressor tuned for low-priced homes, another for mid-range properties, and a simple linear model—each capturing different pricing patterns. Then, a lightweight neural network looks at a listing’s features and outputs a set of weights (summing to one) that tell us how much to trust each expert. The final price is simply the weighted average of their predictions, so the system automatically leans on the right experts for each property, boosting both accuracy and robustness.

4 Implementation

The implementation of our housing price prediction system was modular and reproducible, leveraging Python’s scientific computing stack and several specialized machine learning libraries.

4.1 Software Architecture

Our codebase is structured into separate modules for data preprocessing, model training, evaluation, and visualization. Each machine learning model (Linear Regression, Random Forest, XGBoost, Stacked Model, and Mixture-of-Experts) was encapsulated in its own script or class to ensure reusability and clarity.

4.2 Dependencies and Tools

We used:

- `pandas`, `numpy` for data manipulation.
- `scikit-learn` for standard models (LR, RF), evaluation metrics, and preprocessing.
- `xgboost` for gradient boosting.
- `shap` for model interpretability using SHAP values.
- `matplotlib` for all visualizations.

4.3 Data Pipeline

The preprocessing pipeline handled missing values, encoded categorical features using `LabelEncoder` or `One-Hot Encoding`, and filtered out extreme outliers (e.g., top 30 most expensive properties) to stabilize predictions. We also performed SHAP-based feature selection to reduce dimensionality.

4.4 Model Execution

All models were trained on an 80/20 train-test split with 5-fold cross-validation. Key configurations include:

- **Random Forest:** 300 trees, max depth = 20.
- **XGBoost:** `GridSearchCV`-tuned with early stopping.
- **Stacked Model:** Combined LR and RF as base learners, XGBoost as the final estimator.
- **Mixture-of-Experts:** Neural gate (2 hidden layers, softmax output) dynamically weighted predictions from some number of specialized experts.

4.5 Outputs and Inference

Final predictions were saved to `predicted_prices.csv`. Visualizations included predicted vs actual scatterplots, residuals, SHAP

summary plots, and feature importances for interpretability. All generated plots were automatically stored in structured output folders.

5 Experiments

5.1 Linear Regression, XGBoost, and the Stacked Model

We used the `LinearRegression` and the `StackingRegressor` class from the `scikit-learn` library to create a Linear Regression and Stacked model. For XGBoost we used the `xgboost` library. Numerical variables were extracted from the dataset as features to ensure compatibility. The data was split into training and testing sets (80-20%). We performed k-fold cross-validation (k-fold, k=5) on both the Linear Regression and Stacked Model to evaluate model stability and performance on unseen data. Performance was evaluated using three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

5.2 Random Forest

The Random Forest workflow builds upon the same preprocessed dataset. We import `RandomForestRegressor` with `n_estimators=100` and `random_state=42`, then fit on the training partition. Predictions on the test set are compared against ground truth to compute MAE, RMSE, and R^2 . We generate three diagnostic plots using `matplotlib`: (1) Actual vs. Predicted scatter with a unit line, (2) horizontal bar chart of the top-20 feature importances, and (3) Residuals vs. Predicted scatter to check for heteroscedasticity. SHAP analysis is performed with `shap.TreeExplainer` to produce global summary plots and dependence plots for the top five features.

5.3 K Best Feature Selection Using SHAP Values

We use SHAP values to identify the most influential features in a model, helping us determine the optimal number of top features needed to achieve the best

performance. This approach allows us to retain only the most relevant features while discarding the rest, improving both efficiency and prediction accuracy. To find the optimal number of features, we developed a test suite that evaluates model performance using feature subsets ranging from 10 to 700 in increments of 10. Based on this, we identify a general optimal feature count, and then refine our search by testing within a ± 10 range around that value to pinpoint the true optimum. We assess model performance using the Cross-Validated Root Mean Squared Error (CV-RMSE) with 5-fold cross-validation, where lower scores indicate better results. It’s important to note that `shap.Explainer` does not natively support Linear Regression or Stacked Models. While workarounds exist, they are computationally expensive. As a practical solution for this experiment, we used SHAP values generated by a basic XGBoost regressor for both models.

5.4 Experiment with different datasets

To evaluate the robustness of our improved model, we performed experiments using two distinct datasets:

5.4.1 Housing Prices in Beijing

This dataset was originally presented in the study by Truong et al. [10], which focused on comparing machine learning methods for real estate price prediction. The dataset contains over 230,000 entries with both numerical and categorical features such as location, building structure, floor level, and proximity to subway lines.

5.4.2 California Housing Dataset

As a widely-used benchmark in the machine learning community, we employed the `fetch_california_housing()` dataset from the `sklearn.datasets` module. We compare our results against those presented in a related study on house price prediction available on GitHub². This dataset

²See the work by MYoussef885 at https://github.com/MYoussef885/House_Price_Prediction

contains 20,640 instances and 8 continuous input features representing demographic and geographic information.

6 Results

6.1 ML models results

This section presents the performance outcomes of the five evaluated machine learning models: Linear Regression, Random Forest, XGBoost, Stacked Model and Mixture-of-Experts on the task of predicting housing prices in the Netherlands. The results are based on test set evaluations and are summarized in Table 1.

Table 1: Comparison of Model Performance Metrics

Model	R ²	MAE (€)	RMSE (€)	MAPE (%)
Linear Regression	0.5452	128,013.61	203,447.80	23.6
Random Forest	0.6800	111,700	171,122	17.9
XGBoost	0.7159	108,000	186,000	19.5
Stacked Model	0.6400	116,989.62	180,020.54	18.4
Mixture-of-Experts (MoE)	0.8339	55,833	112,653	8.7

The “Mixture-of-Experts (MoE)” row reports the performance when we bypass the neural gate entirely and simply route each test listing to whichever expert was trained on the price-bin containing its true sale price. In other words, during evaluation we looked up the expert whose training bin matched the listing’s actual price and used that expert’s prediction directly—no learned controller was involved. This gives a reference point for the best possible expert routing before introducing the neural gating layer.

Figure 2 displays the predicted vs. actual prices for the Linear Regression model. The diagonal dashed line represents the ideal prediction.

Overall, the model captures the general trend in housing prices; however, it tends to underestimate high-value properties and shows increased prediction variance in the upper price range.

Figure 3 shows that the Random Forest model improves upon the linear trend captured by Linear Regression, with more accurate predictions and less variance in mid-range housing prices. However, it still underestimates some high-value properties.



Figure 2: Actual vs. Predicted Prices for Linear Regression.

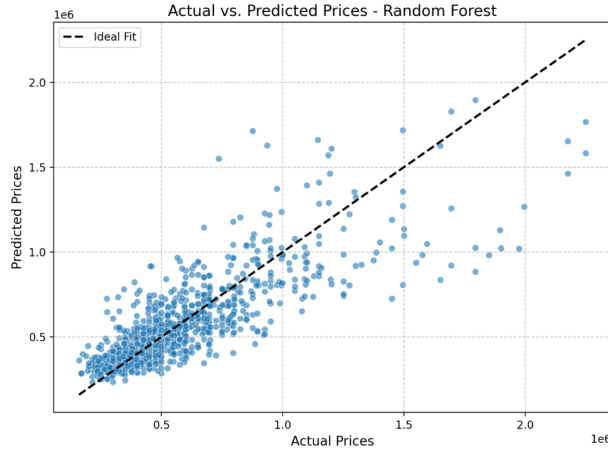


Figure 3: Actual vs. Predicted Prices for Random Forest.

Figure 4 illustrates that the XGBoost model most accurately captures the price distribution, with predictions closely aligning with actual values. While slight underestimation of high-end properties persists, the model demonstrates the least variance and highest predictive accuracy overall.

Figure 5 shows that the stacked model performs poorly with higher prices, vastly underestimating the prices for high-end properties.

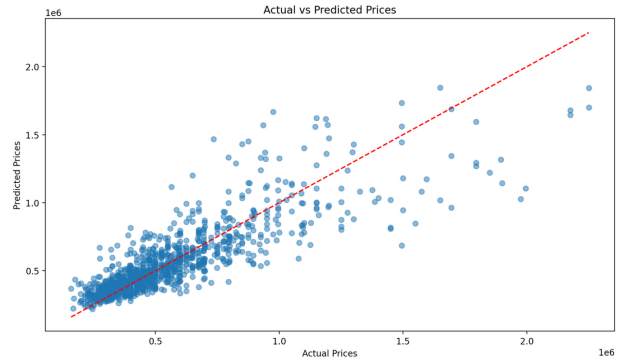


Figure 4: Actual vs. Predicted Prices for XGBoost Model.

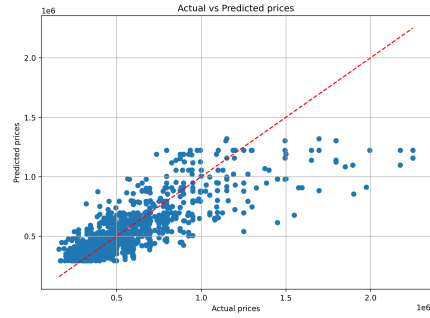


Figure 5: Actual vs. Predicted Prices for the Stacked Model

To interpret feature contributions in the XGBoost model, SHAP (SHapley Additive exPlanations) values were computed. Figure 6 presents the mean absolute SHAP values for the top features. In addition to SHAP analysis for XGBoost, we examined feature importance scores generated by the Random Forest model, as shown in Figure 7.

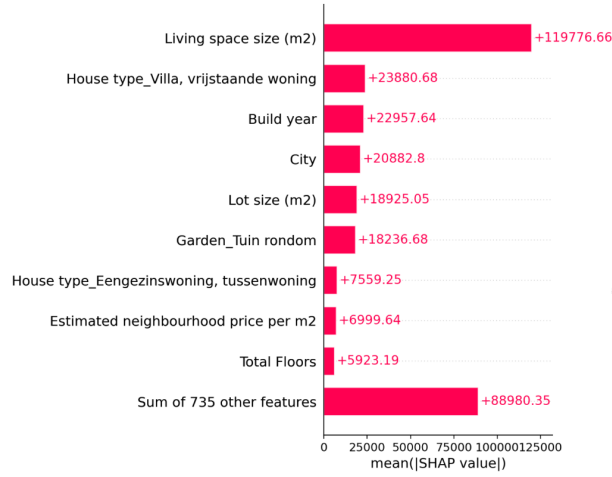


Figure 6: Top SHAP Feature Importances for XGBoost. (Dutch feature names translated: *Vrijstaande woning* = detached house, *Eengezinswoning* = single-family home, *Tuin rondom* = garden around the house, *Tussenwoning* = terraced house)

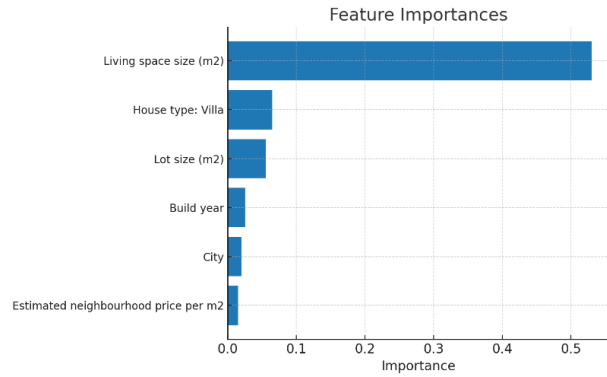


Figure 7: Top Feature Importances in the Random Forest model.

6.2 K Best SHAP Value Feature Selection

The results for using `SelectKBest` show a variance depending on how many features are chosen. Additionally, we can see how it improved each model, reducing the root mean squared errors that are pro-

duced. Below you can see the two Tables 2 and 3 showing the final results. In the appendix you can see Figures 8, 9, 10, 11, 12, 13, 14, and 15 to see how the CV-RMSE evolved for each k

Table 2: A general optimum of how many features are needed for the best results with each model

Model	General K Optimum	CV-RMSE (€)	R ²	MAE (€)	RMSE (€)
Linear Regression	50	198,841.45	0.59	124,126.77	193,364.47
Random Forest	20	180,611.55	0.67	111,642.08	173,771.23
XGBoost	130	182,472.08	0.66	114,299.70	176,621.07
Stacked Model	70	177,941.78	0.65	115,136.14	177,481.01

Table 3: The optimum amount of features selected for each model

Model	Optimum K Value	CV-RMSE (€)	R ²	MAE (€)	RMSE (€)
Linear Regression	52	198,595.03	0.60	123,333.63	191,226.80
Random Forest	26	180,525.53	0.66	112,093.44	174,734.27
XGBoost	130	182,472.08	0.66	114,299.70	176,621.07
Stacked Model	67	177,909.04	0.66	114,778.07	177,084.25

6.3 MoE model results

Table 4 shows the performance comparison of the model with Neural Gating (NG) and without it (base model) on California DataSet.

Table 4: Performance Comparison: With vs. Without Neural Gating for the base model

Metric	Without NG	With NG (MoE)
R^2	0.9572	0.8354
RMSE (€)	23,695	46,389
MAE (€)	15,939	30,747
MAPE (%)	8.2%	8.7%

6.4 Beijing and California datasets

Table 5 compares the performance of different models on the Beijing housing dataset using RMSLE (Root Mean Squared Logarithmic Error) as the metric. Table 6 evaluates model performance on the California housing dataset using R^2 and normalized MAE.

Table 5: Comparison of model performance on Beijing dataset

Dataset	RMSLE
Beijing (XGBoost) [10]	0.14969
Beijing MoE (without NG)	0.1233

Table 6: Comparison of model performance on California datasets. Metrics for MAE are normalized

Dataset	R^2	MAE
California (XGBoost) ²	0.8413	0.30753
MoE (without NG)	0.9572	0.15939
MoE (with NG)	0.8354	0.30747

7 Discussion

Among the baseline models, XGBoost demonstrated the strongest predictive performance, achieving $R^2 = 0.7159$ and $MAE \approx 108k$. Random Forest also showed reasonable accuracy ($R^2 = 0.68$, $MAE \approx 112k$), while Linear Regression struggled with non-linearity, yielding $R^2 = 0.5452$ and the highest overall error.

The stacked ensemble, combining Linear Regression and Random Forest with XGBoost as the meta-learner, did not surpass the performance of the individual models. This may be attributed to a lack of diversity among base learners, overfitting of the final estimator, or limited dataset size.

Evaluation of the Mixture-of-Experts (MoE) framework revealed that the variant without neural gating achieved the highest overall performance ($R^2 = 0.9572$, $MAE = \text{€}15,939$, $MAPE = 8.2\%$). In contrast, the model with neural gating performed worse ($R^2 = 0.8354$, $MAE = \text{€}30,747$), suggesting that the added complexity of neural gating may have caused overfitting in the absence of sufficient data.

To understand the contribution of different features, we analyzed feature importance using Random Forest’s built-in metrics and SHAP values for XGBoost. In both approaches, living space size (m^2) was identified as the most influential factor, followed by property type, construction year, and lot size.

SHAP analysis also demonstrated that a small number of features dominate the model’s predictive power, while the remaining 700+ features had minimal effect. This allowed us to perform effective feature selection, reducing the total number of features from 745 to just $k = 130$, while maintaining competitive performance.

We tested our models on additional datasets from Beijing and California to assess generalizability. On the Beijing dataset, the MoE model without neural gating outperformed the baseline from [10], reducing RMSLE from 0.1497 to 0.1233. On the California dataset, the same model achieved $R^2 = 0.9572$ and normalized $MAE = 0.159$, significantly outperforming the open-source benchmark ($R^2 = 0.8413$, $MAE = 0.3075$).

8 Conclusion

This research explored the application of several machine learning models to predict housing prices in the Netherlands using a multi-city dataset. It was guided by three central questions:

1. How do advanced ensemble models (Stacking, Mixture-of-Experts) compare to traditional models (XGBoost, Random Forest, Linear Regression) in predicting Dutch housing prices?
2. Does the feature selection of the SHAP model correctly represent the most influential factors in Dutch housing price predictions compared to baseline feature selection methods?
3. Which features have the highest impact on Dutch housing prices according to decision tree prediction?

The Mixture-of-Experts without neural gating—outperformed traditional models, achieving $R^2 = 0.9572$, compared to 0.716 for XGBoost and 0.68 for Random Forest. This confirms the effectiveness of expert-based model partitioning over standard boosting or bagging methods. SHAP-based feature selection showed advantages over traditional methods by achieving similar or improved performance with

fewer features, leading to more efficient and interpretable models. Feature importance analysis from both SHAP and Random Forest consistently identified living area, property type, construction year, and city as primary factors in price determination, reflecting the domain expectations confirming their central role in housing price determination.

In conclusion, Mixture-of-Experts offer a powerful and explainable framework for real estate price prediction and SHAP enhances both trust and model transparency. These results could serve as the basis for real-time pricing tools or investment decision systems. Future work could incorporate temporal trends and more specific spatial features to capture additional dynamics in the housing market.

References

- [1] E. Guliker, E. Folmer, and M. van Sinderen, "Spatial determinants of real estate appraisals in the netherlands: A machine-learning approach," *ISPRS International Journal of Geo-Information*, vol. 11, no. 2, p. 125, 2022. DOI: 10.3390/ijgi11020125.
- [2] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*. John Wiley & Sons, 2021.
- [3] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [5] ABN AMRO, *Housing market – building according to need*, Accessed on June 11, 2025, 2025. [Online]. Available: <https://www.abnamro.com/research/en/our-research/housing-market-building-according-to-need>.
- [6] R. Arundel, C. Hochstenbach, and C. Lennartz, "Housing unaffordability and mental health: Dynamics across age and tenure," *International Journal of Housing Policy*, vol. 22, no. 4, pp. 660–681, 2022.
- [7] WOZ-waardeloket — wozwaardeloket.nl, <http://www.wozwaardeloket.nl/>, [Accessed 12-02-2025].
- [8] I. Moreno-Foronda, M.-T. Sánchez-Martínez, and M. Pareja-Eastaway, "Comparative analysis of advanced models for predicting housing prices: A review," *Urban Science*, vol. 9, no. 2, p. 32, 2025.
- [9] Y. Mocquin, *The stacking ensemble method*, Mar. 2024. [Online]. Available: <https://medium.com/data-science/the-stacking-ensemble-method-984f5134463a>.
- [10] D. V. Truong, B. M. Van Nguyen, T.-D. Nguyen, and V.-T. Tran, "A comparison of regression models for predicting housing prices," *Procedia Computer Science*, vol. 176, pp. 760–769, 2020. DOI: 10.1016/j.procs.2020.08.078.

9 Appendix

GitHub page: <https://github.com/Eiragardner/Project2-2>

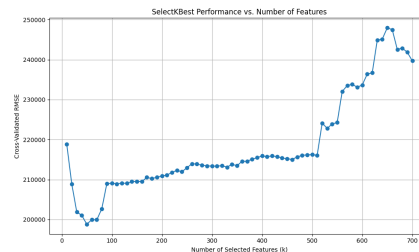


Figure 8: Performance of K Best Features for Linear Regression

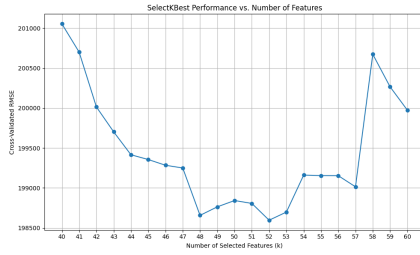


Figure 9: A More Precise K Best Features for Linear Regression

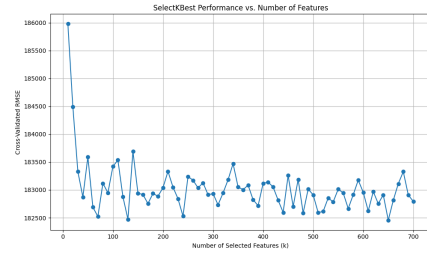


Figure 12: Performance of K Best Features for XG-Boost

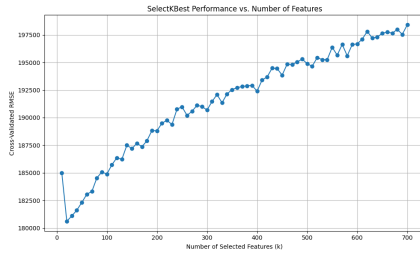


Figure 10: Performance of K Best Features for Random Forest

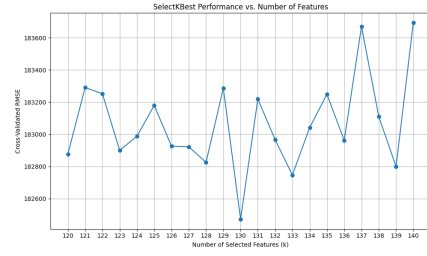


Figure 13: A More Precise K Best Features for XG-Boost

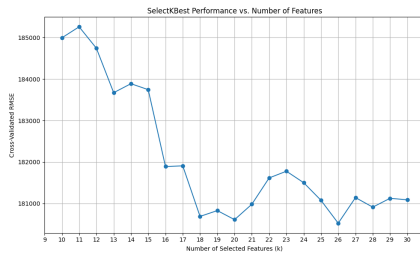


Figure 11: A More Precise K Best Features for Random Forest

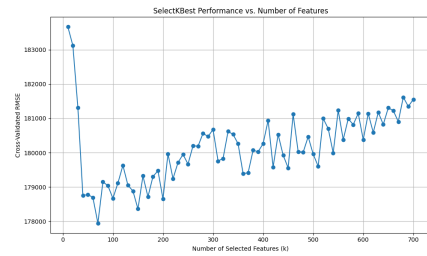


Figure 14: Performance of K Best Features for the Stacked Model

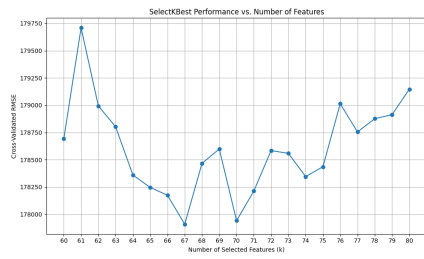


Figure 15: A More Precise K Best Features for the Stacked Model